

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ХОД РАБОТЫ.....	3
1.1 Стенд и материалы	3
1.2 Краткие сведения о malloc/free и аллокаторах.....	4
1.3 Методика измерений	4
1.4 Сравнение времени malloc	5
1.5 malloc с касанием памяти.....	6
1.6 Сравнение времени free.....	6
ЗАКЛЮЧЕНИЕ.....	8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	9
ПРИЛОЖЕНИЕ А Исходный код программы malloc_bench.c	10
ПРИЛОЖЕНИЕ Б Скрипт запуска run_allocators.sh	12
ПРИЛОЖЕНИЕ В Скрипт построения графиков plot_compare.py.....	13

ВВЕДЕНИЕ

Цель работы — исследовать зависимость времени динамического выделения и освобождения памяти от размера запрашиваемого блока в Linux и сравнить несколько реализаций динамического аллокатора на одном экспериментальном стенде.

В качестве усложненного варианта выбран пункт 1 — сравнение с другими malloc. Один и тот же бенчмарк malloc_bench запускался с системным аллокатором glibc и с jemalloc, mimalloc и tcmalloc. Альтернативные реализации подключались через LD_PRELOAD без изменения логики тестовой программы.

Для достижения поставленной цели выполнены следующие действия:

- проведены измерения для 503 размеров блока от 16 байт до 4 МБ;
- для каждого размера выполнены прогревочные и повторные измерительные серии;
- собраны медианные времена malloc, malloc с последующим касанием памяти и free;
- для одинакового набора размеров сопоставлены glibc, jemalloc, mimalloc и tcmalloc;
- по CSV-файлам построены сравнительные графики и сформулированы выводы.

1 ХОД РАБОТЫ

1.1 Стенд и материалы

Эксперимент выполнен в гостевой Ubuntu, запущенной в VirtualBox. Виртуальной машине доступны архитектура x86_64, 2 vCPU и около 4 ГБ оперативной памяти; гостевой системе представлен процессор 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40 GHz. Использовалось ядро 7.0.0-34-generic. Наличие виртуализации и фоновой нагрузки хоста является источником вариации результатов, поэтому абсолютные значения характеризуют именно данный стенд и данную методику.

Таблица 1 — Параметры экспериментального стенда

Параметр	Значение
ОС	Ubuntu в VirtualBox
Архитектура	x86_64
Ядро	7.0.0-34-generic
vCPU	2
Процессор, видимый гостю	Intel Core i5-1135G7 @ 2.40 GHz
ОЗУ	4005488 kB (около 4 ГБ)

В обработку включены четыре файла исходных измерений: glibc.csv, jemalloc.csv, mimalloc.csv и tcmalloc.csv. В каждом файле содержатся 503 строки данных с одинаковым набором размеров и поля size_bytes, malloc_median_ns, malloc_p95_ns, touch_median_ns и free_median_ns. В служебной строке указаны 300 измерительных и 30 прогревочных итераций, максимальный размер — 4194304 байт.

1.2 Краткие сведения о malloc/free и аллокаторах

Функция `malloc(size)` выделяет область памяти размером не менее `size` байт и возвращает указатель на нее либо `NULL` при ошибке. Время вызова зависит не только от размера запроса, но и от внутренних структур конкретного аллокатора: классов размеров, списков свободных блоков, кэшей и стратегии получения новых областей памяти.

Функция `free(ptr)` передает ранее выделенный блок обратно аллокатору. Освобождение не обязательно немедленно приводит к возврату страниц операционной системе: память может остаться во внутренних структурах и затем использоваться повторно.

В работе сравниваются `glibc`, `jemalloc`, `mimalloc` и `tcmalloc`. Для альтернативных библиотек используется `LD_PRELOAD`, поэтому один и тот же исполняемый файл вызывает разные реализации `malloc/free`. Такой подход уменьшает влияние различий в тестовой логике и позволяет сравнивать реализации на одинаковой последовательности размеров.

1.3 Методика измерений

Использован набор из 503 размеров: от 16 до 4096 байт с шагом 16 байт, затем до 64 КБ с шагом 512 байт, до 1 МБ с шагом 8192 байта и несколько дополнительных крупных размеров вплоть до 4 МБ. Перед измерительной серией для каждого размера выполнялось 30 прогревочных итераций, после чего — 300 измерительных.

Основным показателем для выделения памяти выбран `malloc_median_ns` — медиана времени `malloc` в наносекундах. Дополнительно сохранялся 95-й перцентиль `malloc_p95_ns`. Время освобождения представлено полем `free_median_ns`. Медиана используется как устойчивый показатель, менее чувствительный к единичным задержкам виртуальной машины и фоновой нагрузке.

В поле `touch_median_ns` измеряется выделение с последующей записью в полученную область. Этот режим учитывает не только работу `malloc`, но и стоимость первого фактического доступа к памяти. Поэтому его результаты анализируются отдельно от чистого `malloc`.

Все четыре запуска выполнялись одним бинарным файлом. Для `glibc` программа запускалась без `LD_PRELOAD`, для `jemalloc`, `mimalloc` и `tcmalloc` соответствующая динамическая библиотека подставлялась через `LD_PRELOAD`. Графики строились по сохраненным CSV с логарифмическими шкалами по размеру и времени.

1.4 Сравнение времени malloc

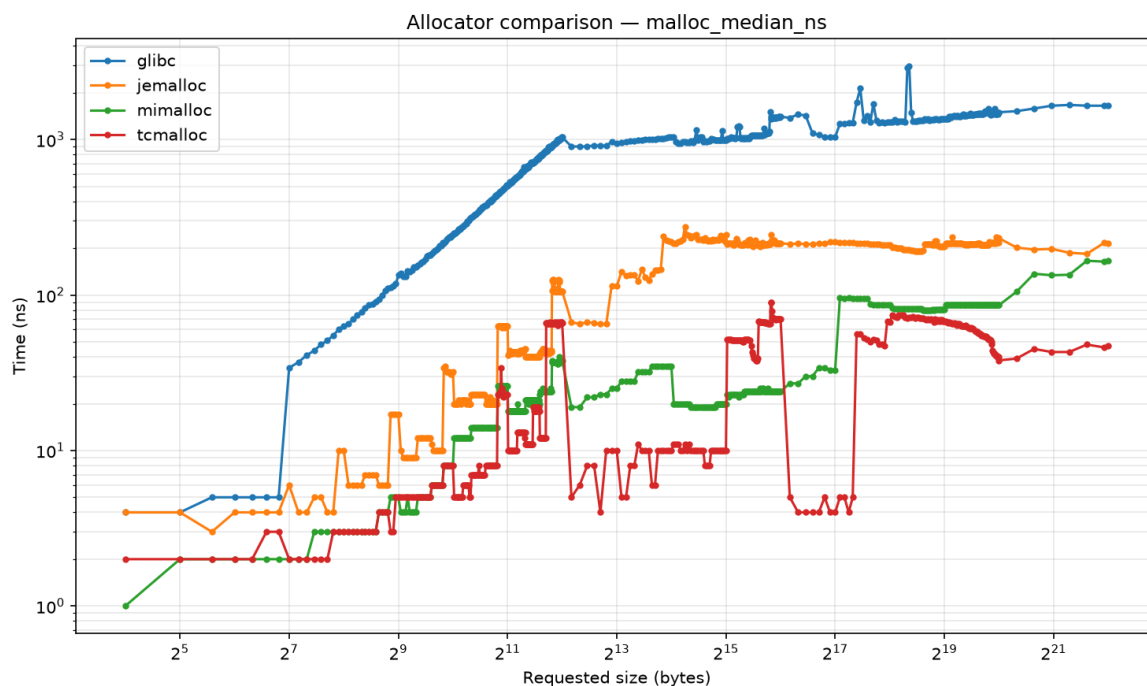


Рисунок 1 — Зависимость медианного времени malloc от размера запроса

На рисунке 1 видны выраженные различия между реализациями. Для glibc после малых размеров наблюдается быстрый рост времени, после чего значения находятся преимущественно около 1 мкс и выше. jemalloc в большей части диапазона располагается ниже glibc. На данном стенде mimalloc и tcmalloc дают меньшие медианные значения на большинстве измеренных размеров; ступени на кривых соответствуют переходам между внутренними классами размеров.

Таблица 2 — Медианное время malloc для выбранных размеров

Размер	glibc, нс	jemalloc, нс	mimalloc, нс	tcmalloc, нс
16 Б	4	4	1	2
4 КБ	1034	106	38	66
64 КБ	1400	215	24	70
1 МБ	1499	232	86	38
4 МБ	1663	215	167	47

Например, при запросе 4 МБ медиана составляет 1663 нс для glibc, 215 нс для jemalloc, 167 нс для mimalloc и 47 нс для tcmalloc. Эти значения не являются универсальной характеристикой библиотек: они зависят от процессора, версий библиотек, состояния кэшей, виртуализации и фоновой нагрузки.

1.5 malloc с касанием памяти

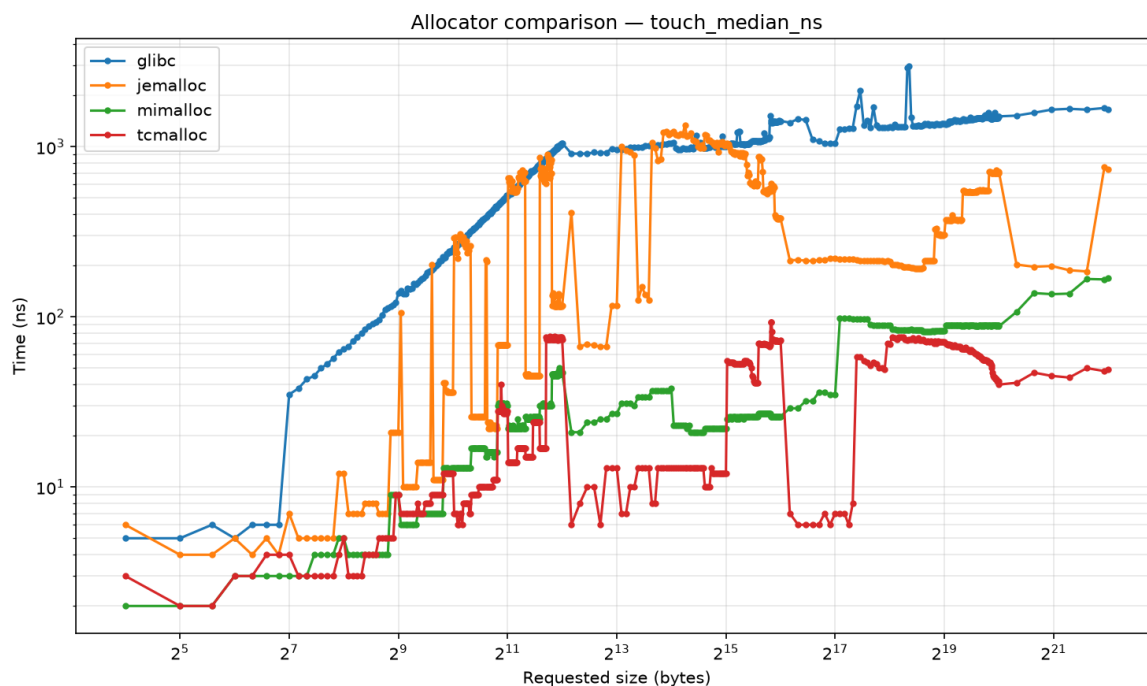


Рисунок 2 — Медианное время malloc с последующим касанием памяти

После добавления доступа к выделенной области часть кривых меняется заметнее. У jemalloc появляются крупные локальные всплески, в том числе на средних и крупных размерах. Для mimalloc и tcmalloc дополнительная стоимость в большинстве точек остается относительно небольшой. У glibc форма кривой в целом близка к результатам чистого malloc.

Таблица 3 — malloc с касанием памяти для выбранных размеров

Размер	glibc, нс	jemalloc, нс	mimalloc, нс	tcmalloc, нс
16 Б	5	6	2	3
4 КБ	1050	116	47	74
64 КБ	1417	379	26	73
1 МБ	1508	710	88	40
4 МБ	1664	737	169	49

Этот режим нельзя трактовать как измерение только аллокатора: первый доступ к выделенной области добавляет стоимость фактического обращения к памяти. Поэтому рисунок 2 дополняет, но не заменяет результаты чистого malloc на рисунке 1.

1.6 Сравнение времени free

CSV также содержат медианное время free. Для краткости характерные точки представлены в таблице 4. На крупных размерах в данном прогоне mimalloc и tcmalloc освобождают блоки быстрее glibc и jemalloc. При этом вызов free обычно возвращает блок

внутреннему аллокатору и не обязательно означает немедленное освобождение соответствующей памяти операционной системой.

Таблица 4 — Медианное время free для выбранных размеров

Размер	glibc, нс	jemalloc, нс	mimalloc, нс	tcmalloc, нс
16 Б	5	16	1	2
4 КБ	148	50	19	15
64 КБ	172	168	19	37
1 МБ	311	421	61	31
4 МБ	540	628	69	33

ЗАКЛЮЧЕНИЕ

В лабораторной работе исследована зависимость времени `malloc/free` от размера запрашиваемой памяти и выполнен выбранный усложненный вариант — сравнение нескольких реализаций `malloc`. Для 503 размеров от 16 байт до 4 МБ использовались 30 прогревочных и 300 измерительных итераций; основным показателем являлась медиана времени в наносекундах.

На конкретном экспериментальном стенде `glibc` имеет наибольшие медианные времена `malloc` в средней и крупной части диапазона. `jemalloc` в большинстве этих точек работает быстрее `glibc`, а меньшие значения на большей части диапазона наблюдаются у `mimalloc` и `tcmalloc`. Для запроса 4 МБ получены 1663 нс для `glibc`, 215 нс для `jemalloc`, 167 нс для `mimalloc` и 47 нс для `tcmalloc`.

Сравнение `free` показывает сходную картину на крупных блоках: для 4 МБ медианы составляют 540, 628, 69 и 33 нс соответственно. Отдельный тест с касанием памяти показывает, что первый доступ к выделенной области способен заметно изменить измеряемое время, поэтому его следует рассматривать отдельно от чистого `malloc`. Поставленная цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Linux man-pages: malloc(3), free(3). URL: <https://man7.org/linux/man-pages/man3/malloc.3.html> (дата обращения: 01.10.2026).
2. Linux man-pages: ld.so(8), переменная LD_PRELOAD. URL: <https://man7.org/linux/man-pages/man8/ld.so.8.html> (дата обращения: 01.10.2026).
3. Linux man-pages: clock_gettime(3). URL: https://man7.org/linux/man-pages/man3/clock_gettime.3.html (дата обращения: 01.10.2026).
4. jemalloc manual. URL: <https://jemalloc.net/jemalloc.3.html> (дата обращения: 01.10.2026).
5. mimalloc: general purpose allocator. URL: <https://github.com/microsoft/mimalloc> (дата обращения: 01.10.2026).
6. gperftools TCMalloc documentation. URL: <https://gperftools.github.io/gperftools/tcmalloc.html> (дата обращения: 01.10.2026).

ПРИЛОЖЕНИЕ А Исходный код программы malloc_bench.c

Ниже приведен исходный код бенчмарка, формирующего CSV с измерениями malloc, malloc с касанием памяти и free.

```
#define _POSIX_C_SOURCE 200809L
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define ITERATIONS 300
#define WARMUP 30
#define BATCH 32
#define MAX_SIZES 503

static uint64_t now_ns(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + (uint64_t)ts.tv_nsec;
}

static int cmp_u64(const void *a, const void *b) {
    const uint64_t x = *(const uint64_t *)a;
    const uint64_t y = *(const uint64_t *)b;
    return (x > y) - (x < y);
}

static uint64_t median_u64(uint64_t *v, size_t n) {
    qsort(v, n, sizeof(v[0]), cmp_u64);
    if (n & 1U)
        return v[n / 2];
    return (v[n / 2 - 1] + v[n / 2]) / 2;
}

static uint64_t p95_u64(uint64_t *v, size_t n) {
    qsort(v, n, sizeof(v[0]), cmp_u64);
    size_t i = (95 * n + 99) / 100;
    if (i == 0) i = 1;
    if (i > n) i = n;
    return v[i - 1];
}

static size_t make_sizes(size_t *sizes) {
    size_t n = 0;
    for (size_t s = 16; s <= 4096; s += 16)
        sizes[n++] = s;
    for (size_t s = 4608; s <= 65536; s += 512)
        sizes[n++] = s;
    for (size_t s = 73728; s <= 1048576; s += 8192)
        sizes[n++] = s;

    const size_t tail[] = {
        1310720, 1638400, 2048000, 2560000,
        3200000, 4000000, 4194304
    };
    for (size_t i = 0; i < sizeof(tail) / sizeof(tail[0]); ++i)
        sizes[n++] = tail[i];
    return n;
}

static int alloc_batch(void **p, size_t size) {
    for (int i = 0; i < BATCH; ++i) {
        p[i] = malloc(size);
        if (p[i] == NULL) {
            for (int j = 0; j < i; ++j) free(p[j]);
            return -1;
        }
    }
    return 0;
}

static void free_batch(void **p) {
    for (int i = 0; i < BATCH; ++i)
        free(p[i]);
}
```

```

static int measure_size(size_t size,
                        uint64_t *malloc_median,
                        uint64_t *malloc_p95,
                        uint64_t *touch_median,
                        uint64_t *free_median) {
    uint64_t tm[ITERATIONS];
    uint64_t tt[ITERATIONS];
    uint64_t tf[ITERATIONS];
    void *p[BATCH];

    for (int w = 0; w < WARMUP; ++w) {
        if (alloc_batch(p, size) != 0) return -1;
        for (int i = 0; i < BATCH; ++i)
            ((volatile unsigned char *)p[i])[0] = (unsigned char)i;
        free_batch(p);
    }

    for (int r = 0; r < ITERATIONS; ++r) {
        uint64_t t0 = now_ns();
        if (alloc_batch(p, size) != 0) return -1;
        uint64_t t1 = now_ns();
        tm[r] = (t1 - t0) / BATCH;

        t0 = now_ns();
        free_batch(p);
        t1 = now_ns();
        tf[r] = (t1 - t0) / BATCH;

        t0 = now_ns();
        if (alloc_batch(p, size) != 0) return -1;
        for (int i = 0; i < BATCH; ++i)
            ((volatile unsigned char *)p[i])[0] = (unsigned char)(r + i);
        t1 = now_ns();
        tt[r] = (t1 - t0) / BATCH;
        free_batch(p);
    }

    uint64_t tm_copy[ITERATIONS];
    memcpy(tm_copy, tm, sizeof(tm));
    *malloc_median = median_u64(tm, ITERATIONS);
    *malloc_p95 = p95_u64(tm_copy, ITERATIONS);
    *touch_median = median_u64(tt, ITERATIONS);
    *free_median = median_u64(tf, ITERATIONS);
    return 0;
}

int main(void) {
    size_t sizes[MAX_SIZES];
    const size_t n = make_sizes(sizes);
    const char *name = getenv("ALLOCATOR_NAME");
    if (name == NULL || *name == '\0') name = "unknown";

    printf("# allocator=%s iterations=%d warmup=%d max_size=%zu\n",
           name, ITERATIONS, WARMUP, sizes[n - 1]);
    printf("size_bytes,malloc_median_ns,malloc_p95_ns,"
           "touch_median_ns,free_median_ns\n");

    for (size_t i = 0; i < n; ++i) {
        uint64_t mm, p95, touch, fr;
        if (measure_size(sizes[i], &mm, &p95, &touch, &fr) != 0) {
            fprintf(stderr, "allocation failed for %zu bytes\n", sizes[i]);
            return 1;
        }
        printf("%zu,%llu,%llu,%llu,%llu\n",
               sizes[i],
               (unsigned long long)mm,
               (unsigned long long)p95,
               (unsigned long long)touch,
               (unsigned long long)fr);
        fflush(stdout);
    }
    return 0;
}

```

ПРИЛОЖЕНИЕ Б Скрипт запуска run_allocators.sh

Скрипт собирает один бинарный файл, запускает его с glibc и с альтернативными аллокаторами через LD_PRELOAD, после чего строит графики.

```
#!/usr/bin/env bash
set -euo pipefail

CC=${CC:-gcc}
CFLAGS=${CFLAGS:--O2 -std=c11 -Wall -Wextra}

$CC $CFLAGS -o malloc_bench malloc_bench.c

find_so() {
    local pattern="$1"
    ldconfig -p | awk -v p="$pattern" '$1 ~ p { print $NF; exit }'
}

run_one() {
    local name="$1"
    local so="${2:-}"
    echo "Running ${name}..." >&2
    if [[ -n "$so" ]]; then
        ALLOCATOR_NAME="$name" LD_PRELOAD="$so" \
            ./malloc_bench > "${name}.csv"
    else
        ALLOCATOR_NAME="$name" ./malloc_bench > "${name}.csv"
    fi
}

JEMALLOC=$(find_so '^libjemalloc\.so')
MIMALLOC=$(find_so '^libmimalloc\.so')
TCMALLOC=$(find_so '^libtcmalloc(_minimal)?\.so')

[[ -n "$JEMALLOC" ]] || { echo "jemalloc not found" >&2; exit 1; }
[[ -n "$MIMALLOC" ]] || { echo "mimalloc not found" >&2; exit 1; }
[[ -n "$TCMALLOC" ]] || { echo "tcmalloc not found" >&2; exit 1; }

run_one glibc
run_one jemalloc "$JEMALLOC"
run_one mimalloc "$MIMALLOC"
run_one tcmalloc "$TCMALLOC"

python3 plot_compare.py
```

ПРИЛОЖЕНИЕ В Скрипт построения графиков plot_compare.py

Скрипт читает четыре CSV и формирует два сравнительных графика, использованных в отчете.

```
#!/usr/bin/env python3
import pandas as pd
import matplotlib.pyplot as plt

FILES = {
    "glibc": "glibc.csv",
    "jemalloc": "jemalloc.csv",
    "mimalloc": "mimalloc.csv",
    "tcmalloc": "tcmalloc.csv",
}

SERIES = [
    (
        "malloc_median_ns",
        "Allocator comparison - malloc_median_ns",
        "compare_malloc_median_ns.png",
    ),
    (
        "touch_median_ns",
        "Allocator comparison - touch_median_ns",
        "compare_touch_median_ns.png",
    ),
]

def load(path):
    return pd.read_csv(path, comment="#")

frames = {name: load(path) for name, path in FILES.items()}

for metric, title, output in SERIES:
    plt.figure(figsize=(10, 6))
    for name, df in frames.items():
        plt.plot(
            df["size_bytes"],
            df[metric],
            marker="o",
            markersize=2.5,
            linewidth=1.3,
            label=name,
        )

    plt.xscale("log", base=2)
    plt.yscale("log")
    plt.xlabel("Requested size (bytes)")
    plt.ylabel("Time (ns)")
    plt.title(title)
    plt.grid(True, which="both", alpha=0.25)
    plt.legend()
    plt.tight_layout()
    plt.savefig(output, dpi=180)
    plt.close()
```